Assignment #2: Machine Learning Exercise

Clustering

Data preparation and preprocessing

Importance of Data Standardization

Data standardization ensures that all features are on the same scale so that no single feature dominates others due to larger numerical values. This is important for algorithms like K-Means, EM and neural networks that rely on distance calculations or gradient based optimization. Without standardization features with larger magnitudes could cause a bias in the results and lead to poor model performance.

```
C:\Users\alexi\OneDrive\Desktop\Assignment_2_ML\.venv\Scripts\python.exe C:\Us
Before:
                T           P        TC          SV
mean  303.150000  121.100000  0.034696  415.243807
std    34.156543   95.286764  0.004071   22.532554

After:
                  T             P            TC            SV
mean -3.256857e-15 -6.929483e-17  1.039423e-16 -1.827651e-15
std   1.000000e+00  1.000000e+00  1.000000e+00  1.000000e+00

Process finished with exit code 0
```

Before Standardization

Before standardization, the features had different means and standard deviations. The means were not centered around zero, and the standard deviations varied across features.

This means that the features were on different scales which can negatively impact clustering and classification performance.

After Standardization

After applying z-score standardization the mean of each feature is approximately 0 and the standard deviation is approximately 1. This confirms that all features have been normalized to the same scale, this ensures equal contribution during model training and improving the effectiveness of machine learning algorithms.

K-means clustering

K-Means Clustering Results (K = 3)

Initialization Method

K-Means was initialized using manual selection of centroids from the standardized dataset. Three data points were randomly selected using a fixed random seed to ensure reproducibility. Manual initialization was chosen to allow explicit reporting and control of the initial centroids, making the clustering process more transparent compared to methods such as K-Means++, which select centroids probabilistically.

Initial Centroids

- Cluster 0: [0.29276967, -0.95606143, -0.01069514, 0.67715827]
- Cluster 1: [-1.46384837, 1.87749057, 0.61708479, -2.02420718]
- Cluster 2: [1.46384837, -0.95606143, 1.22633744, 1.32966080]

The initial centroids were selected from the standardized dataset and show varied starting positions across the feature space, including both positive and negative values, ensuring the algorithm begins with diverse cluster seeds.

Number of Iterations Until Convergence

The algorithm converged after 13 iterations.

Final Centroids

- Cluster 0: [-0.91963238, -0.64060995, -1.12229501, -0.67103296]
- Cluster 1: [-0.64546080, 1.20766756, -0.10592672, -0.97164177]
- Cluster 2: [0.91664782, -0.10746251, 0.81176594, 0.89421860]

The final centroids show that Cluster 2 has mostly positive values indicating higher than average features while Clusters 0 and 1 have more negative values meaning lower or mixed feature patterns. This indicates that the data has been grouped into distinct sections based on feature intensity.

Cluster Variance (WCSS)

The within-cluster sum of squares (WCSS) is: 575527.9168828558

Number of Samples per Cluster

- Cluster 0: 134,440
- Cluster 1: 88,712
- Cluster 2: 196,848

```
Initialization method: manual selection from standardized dataset

Initial centroids:
Cluster 0: [ 0.29276967 -0.95606143 -0.01069514  0.67715827]
Cluster 1: [-1.46384837  1.87749057  0.61708479 -2.02420718]
Cluster 2: [ 1.46384837 -0.95606143  1.22633744  1.3296608 ]

Number of iterations until convergence: 13

Final centroids:
Cluster 0: [-0.91963238 -0.64060995 -1.12229501 -0.67103296]
Cluster 1: [-0.6454608   1.20766756 -0.10592672 -0.97164177]
Cluster 2: [ 0.91664782 -0.10746251  0.81176594  0.8942186 ]

Cluster variances (Within-Cluster Sum of Squares):
575527.9168828558

Number of samples assigned to each cluster:
Cluster 0: 134440
Cluster 1: 88712
Cluster 2: 196848

Process finished with exit code 0
```

EM Clustering Results (GMM, K = 3)

Initialization Method and Initial Parameters

The Gaussian Mixture Model was initialized using the K-Means initialization method. The model was configured with 3 mixture components, a full covariance matrix, a maximum of 300 iterations, a convergence tolerance of 0.001 and a fixed random seed to ensure reproducibility.

Convergence Criterion

The convergence criterion was based on both the tolerance and the maximum number of iterations. The tolerance was set to 0.001 to balance convergence accuracy and computational efficiency. The maximum number of iterations was set to 300. The model successfully converged after 22 iterations.

Covariance Type

The covariance type used was full. This allows each cluster to have its own shape, size, and orientation making it more flexible than diagonal or spherical covariance structures. Since the standardized features may still show correlations, using a full covariance matrix enables the model to better capture the underlying structure of the data.

Mixture Weights

Cluster 0: 0.3749446801094954
Cluster 1: 0.3887166501796879
Cluster 2: 0.23633866971081668

The mixture weights represent the proportion of data points assigned to each cluster. Cluster 0 and Cluster 1 contain most of the data while Cluster 2 is smaller. This indicates that the data is more concentrated in the first two clusters with the third cluster representing a less frequent pattern in the dataset.

Means

Cluster 0: [1.02910558, -0.18180877, 0.91829830, 1.02764737]
Cluster 1: [-0.61888768, 0.74748636, -0.27402513, -0.80843829]
Cluster 2: [-0.61475320, -0.94089932, -1.00615330, -0.31382084]

The means represent the center of each cluster and the typical feature values within it. Positive values indicate above average features and negative values indicate below average features. Cluster 0 corresponds to higher values, Cluster 2 to lower values and Cluster 1 shows a mixed pattern.

Covariances

Cluster 0:
[[0.19232822, 0.06209193, 0.22937125, 0.14216375],
[0.06209193, 0.72126723, 0.17829830, -0.17642619],
[0.22937125, 0.17829830, 0.29269816, 0.14493192],
[0.14216375, -0.17642619, 0.14493192, 0.20417809]]

Cluster 1:
[[0.52780882, 0.22263247, 0.42842636, 0.42449766],
[0.22263247, 0.73680460, 0.45764267, -0.03057272],
[0.42842636, 0.45764267, 0.55078686, 0.24300033],
[0.42449766, -0.03057272, 0.24300033, 0.42604431]]

Cluster 2:
[[0.36973862, 0.01455467, 0.37622722, 0.35944164],
[0.01455467, 0.01811475, 0.01951813, 0.01186976],
[0.37622722, 0.01951813, 0.37460002, 0.37800725],
[0.35944164, 0.01186976, 0.37800725, 0.37789631]]

The covariance matrices describe the spread and shape of each cluster. Diagonal values show feature variance while off diagonal values capture relationships between features. The differences across clusters indicate that each cluster has a unique structure justifying the use of full covariance.

Probabilities p(z = k | x) for 3 Samples

Sample 0: [1.48470919e-19, 3.15674309e-03, 9.96843256e-01]
Sample 1: [3.05577795e-25, 1.65113758e-03, 9.98348862e-01]
Sample 2: [1.39630362e-27, 3.43708850e-04, 9.99656292e-01]

The probabilities represent the likelihood of each sample belonging to each cluster. All three samples have probabilities above 0.996 for Cluster 2 indicating very confident assignments with little ambiguity.

```
EM Clustering Results (GMM, K=3)

Initialization method: kmeans

Convergence criterion:
Tolerance: 0.001
Maximum iterations: 300
Converged: True
Number of iterations until convergence: 22

Covariance type:
full

Mixture weights:
Cluster 0: 0.3749446801094954
Cluster 1: 0.3887166501796879
Cluster 2: 0.23633866971081668

Means:
Cluster 0: [ 1.02910558 -0.18180877  0.9182983   1.02764737]
Cluster 1: [-0.61887678  0.74748636 -0.27402513 -0.80043629]
Cluster 2: [-0.6147532  -0.94098932 -1.0061533  -0.31382084]

Covariances:

Cluster 0:
[[ 0.19232822  0.06209193  0.22937125  0.14216375]
 [ 0.06209193  0.72126723  0.1782983  -0.17642619]
 [ 0.22937125  0.1782983   0.29269816  0.14493192]
 [ 0.14216375 -0.17642619  0.14493192  0.20417809]]

Cluster 1:
[[ 0.52780828  0.22263247  0.42842636  0.42449766]
 [ 0.22263247  0.73686046  0.45764267 -0.03057272]
 [ 0.42842636  0.45764267  0.55070686  0.24300303]
 [ 0.42449766 -0.03057272  0.24300303  0.42664413]]

Cluster 2:
[[0.36993862 0.01453467 0.37622722 0.35964164]
 [0.01453467 0.01811475 0.01951813 0.01186976]
 [0.37622722 0.01951813 0.38740602 0.37080725]
 [0.35964164 0.01186976 0.37080725 0.37789063]]

Probabilities p(z = k | x) for 3 samples:
Sample 0: [1.48479019e-19 3.15674390e-03 9.96843256e-01]
Sample 1: [3.05577793e-25 1.65113758e-03 9.98348862e-01]
Sample 2: [1.39630302e-27 3.43708058e-04 9.99656292e-01]

Process finished with exit code 0
```

Self-Organizing Map (SOM) Clustering Results

Grid Size

A 15 × 15 SOM grid was used, resulting in 225 neurons. This grid was chosen because it provides enough neurons to capture a meaningful structure and local patterns in the standardized dataset without making the map unnecessarily large or computationally expensive. It offers a good balance between representation detail and training efficiency for four-feature input data.

Neighborhood Function

The neighborhood function used was Gaussian. This function updates not only the best matching unit but also nearby neurons allowing the SOM to preserve topological

relationships in the data and produce a smoother organization of prototypes across the 2D grid.

Learning Rate Schedule

The SOM used an initial learning rate of 0.5. This learning rate allows the SOM to make meaningful updates during training while remaining stable enough to gradually organize the map. As training progresses the effective updates decrease helping the map converge.

Number of Iterations

The SOM was trained for 1000 iterations.

Similarity Metric Used

The similarity metric used was Euclidean distance. This is appropriate because the data was standardized before training, so all four features contribute on the same scale when comparing input samples to neuron weight vectors.

Neuron Weight Vectors (Prototypes)

After training, the neuron weight vectors were extracted from the SOM as prototypes. Since the SOM grid size was 15 × 15 and each neuron has 4 weights, the prototype matrix had shape (225, 4). These prototypes represent the learned feature-space positions of the SOM neurons.

Clustering SOM Prototypes with K-Means

Because SOM does not directly output cluster labels, the extracted neuron prototypes were clustered using K-Means with K = 3. This grouped the prototypes into three clusters, which were then used to assign each data point to a final cluster based on the cluster of its BMU.

Final Cluster Centroids

The final cluster centroids obtained from K-Means on the SOM prototypes were:

Cluster 0: [0.8463, -0.2214, 0.7249, 0.8979]
Cluster 1: [-1.3270, -0.9278, -1.6566, -1.0208]
Cluster 2: [-0.4058, 0.2013, -0.3403, -0.4415]

The final centroids show that Cluster 0 has mostly positive values, indicating higher than average feature values while Cluster 1 has strongly negative values representing lower than average features. Cluster 2 contains values closer to zero indicating more moderate feature

patterns. This suggests that the SOM prototypes capture distinct regions of the feature space.

Final Cluster Sizes

After assigning each data point to the cluster of its BMU, the number of samples in each cluster was:

Cluster 0: 186743
Cluster 1: 81726
Cluster 2: 151531

The cluster sizes show that Cluster 0 is the largest followed by Cluster 2 while Cluster 1 is the smallest. This indicates that higher and moderate feature patterns are more common in the dataset while the low feature pattern represented by Cluster 1 is less frequent.

```
SOM Clustering Results
--------------------------------------------------
Grid size:
15x15

Neighborhood function:
gaussian

Learning rate schedule:
Initial learning rate = 0.5

Number of iterations:
1000

Similarity metric used:
Euclidean distance

Neuron weight vectors (prototypes):
Prototype matrix shape: (225, 4)
First 5 prototypes:
[[-1.4638 -1.0851 -1.7924 -0.7767]
 [-1.4638 -1.0719 -1.7673 -0.8133]
 [ 0.5906  0.9939  0.6632  0.2194]
 [ 0.8518 -0.75    0.6468  1.1456]
 [-0.2933 -0.7463 -0.5764 -0.0126]]

K-Means on SOM prototypes:
Number of prototype clusters: 3

Final cluster centroids:
Cluster 0: [ 0.8463 -0.2214  0.7249  0.8979]
Cluster 1: [-1.327  -0.9278 -1.6566 -1.0208]
Cluster 2: [-0.4058  0.2013 -0.3403 -0.4415]

Cluster sizes (after assigning each data point to BMU cluster):
Cluster 0: 186743
Cluster 1: 81726
Cluster 2: 151531

Process finished with exit code 0
```

Cluster Evaluation and Interpretation

Silhouette Score Analysis

Silhouette scores were computed using a random sample of 10,000 data points to evaluate clustering quality. The scores were:

K-Means: 0.4211
EM: 0.3318
SOM-derived: 0.3533

The silhouette score measures how well-separated clusters are, with values closer to 1 indicating better separation. Among the three methods, K-Means achieved the highest score, indicating the best cluster separation. The EM model produced the lowest score, suggesting more overlap between clusters, while the SOM-derived clustering performed moderately but did not outperform K-Means.

Wobbe Index to Quality Classes

The Wobbe Index was converted into three quality classes: Regular, Medium, and Premium. The thresholds were determined using the 33rd and 66th percentiles of the data, ensuring an approximately equal distribution of samples across the three classes.

Regular: ≤ 53.8898
Medium: ≤ 54.1174
Premium: > 54.1174

This percentile-based approach provides a balanced and unbiased way to define class boundaries without relying on arbitrary thresholds.

Feature Means and Variances per Class

Mean values for each class:

Medium: [0.001443, 0.004263, -0.008862, -0.005114]
Premium: [0.000523, -0.002988, -0.002642, -0.141913]
Regular: [-0.001981, -0.001185, 0.011583, 0.151327]

Variance values for each class:

Medium: [0.998809, 1.001954, 1.000528, 0.981893]
Premium: [1.001423, 1.000931, 0.967988, 0.948534]
Regular: [0.999734, 0.997072, 1.032248, 1.027471]

The means are close to zero due to standardization, and variances are near one, confirming proper scaling. Differences between class means indicate subtle distinctions between quality levels.

Adjusted Rand Index (ARI) Analysis

K-Means: 0.0003
GMM: 0.0012
SOM-derived: 0.0002

The Adjusted Rand Index evaluates how well clustering results align with the true class labels. All ARI values are close to zero, indicating that none of the clustering methods strongly align with the defined quality classes. This suggests that the natural clustering structure of the features does not directly correspond to the Wobbe Index-based classification.

Interpretation

K-Means produced the best cluster separation based on silhouette score, while GMM and SOM showed weaker separation. However, all methods had very low ARI values, indicating poor alignment with the true quality classes. This suggests that while clustering can group similar data points, the resulting clusters do not directly correspond to the predefined gas quality categories based on the Wobbe Index.

```
Silhouette Scores (10k samples):
K-Means: 0.4211
EM (GMM): 0.3318
SOM-derived: 0.3533

Wobbe Index thresholds:
Regular: <= 53.8898
Medium: <= 54.1174
Premium: > 54.1174

Mean of features for each class:
                T         P        TC        SV
Quality
Medium   0.001443  0.004263 -0.008862 -0.005114
Premium  0.000523 -0.002988 -0.002642 -0.141913
Regular -0.001981 -0.001185  0.011583  0.151327

Variance of features for each class:
                T         P        TC        SV
Quality
Medium   0.998809  1.001954  1.000528  0.981893
Premium  1.001423  1.000931  0.967988  0.948534
Regular  0.999734  0.997072  1.032248  1.027471

Adjusted Rand Index (ARI):
K-Means: 0.0003
GMM: 0.0012
SOM-derived: 0.0002

Process finished with exit code 0
```

Classification

MLP Classifier Results

Topology (Layers and Number of Neurons)

The MLP classifier used a topology with two hidden layers. The first hidden layer contained 32 neurons, and the second hidden layer contained 16 neurons. The input layer used the four standardized chemical features, and the output layer predicted the three quality classes: Regular, Medium, and Premium.

Activation Function

The activation function used in the hidden layers was ReLU. It was chosen because it is computationally efficient and helps the network learn nonlinear relationships in the data.

Optimizer (Gradient Method)

The optimizer used was Adam. Adam is a gradient-based optimization method that combines adaptive learning rates with momentum, making it effective and stable for training neural networks.

Learning Rate

The initial learning rate was set to 0.001. This is a commonly used value that provides a good balance between stable convergence and efficient learning.

Number of Epochs

The model was trained for a maximum of 225 epochs.

Training Method and Batch Size

The model was trained using mini-batch gradient descent through the Adam optimizer, with a batch size of 256. This allows the network to update weights efficiently while maintaining stable learning.

Regularization Method

The regularization method used was L2 regularization, with alpha = 0.0001. This helps reduce overfitting by discouraging overly large weight values.

Validation Accuracy

The validation accuracy obtained was 0.8117, or 81.17%.

Confusion Matrix (Test Set)

The confusion matrix on the test set was:

[[18668, 2122, 0],
[2480, 14658, 3652],
[0, 3902, 17518]]

There is noticeable confusion between the Medium class and the other two classes, particularly with misclassifications into both Regular and Premium. This suggests that Medium represents a more ambiguous region in the feature space.

Test Accuracy

The MLP achieved a test accuracy of 80.70%.

Test F1-Score

The macro F1-score on the test set was 0.8066.

Classification Report

For the Regular class, the model achieved precision 0.88, recall 0.90, and F1-score 0.89.
For the Medium class, the model achieved precision 0.71, recall 0.71, and F1-score 0.71.
For the Premium class, the model achieved precision 0.83, recall 0.82, and F1-score 0.82.

Overall, the MLP classifier performed well, with the strongest performance on the Regular and Premium classes, while the Medium class was more difficult to classify accurately.

```
C:\Users\alexi\OneDrive\Desktop\Assignment_2_ML\.venv\Scripts\python.exe C:\Users\alexi\OneDrive\Desktop\Assignment_2_ML\mlp_classifier.py
Validation Accuracy: 0.8117

MLP Test Confusion Matrix:
[[18668  2122     0]
 [ 2480 14658  3652]
 [    0  3902 17518]]

MLP Test Accuracy:
80.7 %

MLP Test F1-score (macro):
0.8066

Classification Report:
              precision    recall  f1-score   support

     Regular       0.88      0.90      0.89     20790
      Medium       0.71      0.71      0.71     20790
     Premium       0.83      0.82      0.82     21420

    accuracy                           0.81     63000
   macro avg       0.81      0.81      0.81     63000
weighted avg       0.81      0.81      0.81     63000


Process finished with exit code 0
```

RBF Classifier Results

The RBF classifier was implemented using K-Means centroids as the RBF centers. The number of hidden units was set to 150, meaning 150 cluster centroids were used as radial basis centers. Smaller configurations with 3 and 50 hidden units were also evaluated, yielding validation accuracies of approximately 34.98% and 65.84%. Using 150 hidden units achieved the best validation accuracy of approximately 69.5%, indicating that a larger number of centers was necessary to better capture the structure of the data.

The kernel width was computed as the average pairwise distance between the selected K-Means centers. The resulting kernel width was:

$\sigma = 2.503$

Confusion Matrix (Test Set)

[[16544, 4105, 141],
[4657, 10112, 6021],
[520, 3950, 16950]]

The Medium class is frequently misclassified as both Regular and Premium indicating that the model struggles to distinguish this class from the others.

Test Accuracy

The RBF classifier achieved a test accuracy of 69.22%.

Test F1-Score

The macro F1-score on the test set was 0.6862.

Classification Report

For the Regular class, the model achieved precision 0.76, recall 0.80, and F1-score 0.78.
For the Medium class, the model achieved precision 0.56, recall 0.49, and F1-score 0.52.
For the Premium class, the model achieved precision 0.73, recall 0.79, and F1-score 0.76.

The classification shows that the Medium class has the weakest performance, with lower precision and recall confirming that it is the most difficult class to classify. However, the Regular and Premium classes achieve higher and more consistent performance.

```
Validation Accuracy: 0.6951

RBF Test Confusion Matrix:
[[16544  4105   141]
 [ 4657 10112  6021]
 [  520  3950 16950]]

RBF Test Accuracy:
69.22 %

RBF Test F1-score (macro):
0.6862

Classification Report:
              precision    recall  f1-score   support

     Regular       0.76      0.80      0.78     20790
      Medium       0.56      0.49      0.52     20790
     Premium       0.73      0.79      0.76     21420

    accuracy                           0.69     63000
   macro avg       0.68      0.69      0.69     63000
weighted avg       0.68      0.69      0.69     63000


RBF Parameters:
Kernel width (sigma): 2.503
Number of hidden units: 150

Process finished with exit code 0
```

Model Comparison (MLP vs RBF)

Which model was easier to tune?

The MLP classifier was easier to tune. It relies on a relatively small set of hyperparameters such as the number of hidden layers, number of neurons, learning rate, and batch size. In contrast, the RBF classifier required tuning the number of centers (hidden units) and the kernel width (σ), both of which significantly impacted performance. Selecting an appropriate number of centers required multiple experiments and increased computational cost, making RBF more difficult to tune.

Which model performed better?

The MLP classifier performed better overall. It achieved a higher test accuracy (80.70%) and a higher macro F1-score (0.8066) compared to the RBF classifier, which achieved 69.22% accuracy and 0.6862 F1-score. This indicates that the MLP was more effective at capturing the relationships between the features and the target classes.

Why might their behaviors differ?

The difference in performance is due to how each model learns patterns in the data. The MLP is a deep learning model that learns complex nonlinear relationships through multiple layers and adaptive weight updates, allowing it to generalize well across the dataset. On the other hand, the RBF classifier relies on distance-based transformations using fixed centers, which makes it more sensitive to the placement and number of those centers. If the centers are not optimally positioned or insufficient in number, the RBF model may struggle to capture complex decision boundaries. Additionally, the MLP can learn global patterns across the entire dataset, while the RBF model focuses more on local regions around its centers, which can limit its performance on more complex data distributions.